

MCS-011: Problem Solving and Programming (C)

Q1. Call-by-Value vs Call-by-Reference with Swap

- Differences:
 1. Call-by-value: function gets copies; changes don't affect caller.
 2. Call-by-reference: function gets addresses; changes reflect in caller.
- Programs:

```
// Call-by-Value Swap (no effect outside)
#include <stdio.h>
void swap_val(int a, int b){
    int t=a; a=b; b=t;
}
int main(){
    int x=10,y=20;
    printf("Before: x=%d y=%d\n",x,y);
    swap_val(x,y);
    printf("After (call-by-value): x=%d y=%d\n",x,y);
    return 0;
}

// Call-by-Reference Swap (using pointers)
#include <stdio.h>
void swap_ref(int *a, int *b){
    int t=*a; *a=*b; *b=t;
}
int main(){
    int x=10,y=20;
    printf("Before: x=%d y=%d\n",x,y);
    swap_ref(&x,&y);
    printf("After (call-by-reference): x=%d y=%d\n",x,y);
    return 0;
}
```

- Answer: Call-by-reference reflects changes outside the function because it operates on the original variables via their addresses.
-

Q2. Average, Highest, Lowest using Array

```
#include <stdio.h>
int main(){
    int n; printf("Enter number of students: ");
    if(scanf("%d",&n)!=1 || n<=0){ printf("Invalid n"); return 0; }
    float a[n], sum=0, min, max;
    for(int i=0;i<n;i++){
        printf("Mark %d: ",i+1); scanf("%f",&a[i]);
        sum+=a[i];
        if(i==0){ min=max=a[i]; }
    }
```

```

        else { if(a[i]<min) min=a[i]; if(a[i]>max) max=a[i]; }
    }
    printf("Average=%.2f\nHighest=%.2f\nLowest=%.2f\n", sum/n, max, min);
    return 0;
}

```

Q3. Matrix Multiply N×N and Check Diagonal

```

#include <stdio.h>
int main(){
    int N; printf("Enter N: "); scanf("%d",&N);
    if(N<=0||N>50){ printf("Invalid N"); return 0; }
    int A[50][50],B[50][50],C[50][50]={0};
    for(int i=0;i<N;i++)for(int j=0;j<N;j++){ printf("A[%d][%d]: ",i,j);
scanf("%d",&A[i][j]); }
    for(int i=0;i<N;i++)for(int j=0;j<N;j++){ printf("B[%d][%d]: ",i,j);
scanf("%d",&B[i][j]); }
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++)
            for(int k=0;k<N;k++)
                C[i][j]+=A[i][k]*B[k][j];
    int diagonal=1;
    for(int i=0;i<N;i++)
        for(int j=0;j<N;j++)
            if(i!=j && C[i][j]!=0) diagonal=0;
    printf("Result C:\n");
    for(int i=0;i<N;i++){ for(int j=0;j<N;j++) printf("%d ",C[i][j]);
printf("\n"); }
    printf("C is %sdiagonal matrix\n", diagonal?"":"not ");
    return 0;
}

```

Q4. Menu-driven String Operations (no built-ins)

```

#include <stdio.h>

int len(const char*s){ int i=0; while(s[i]!='\0') i++; return i; }
int is_vowel(char c){ char x=(c>='A'&&c<='Z')?c+32:c; return
x=='a'||x=='e'||x=='i'||x=='o'||x=='u'; }
int is_pal(const char*s){
    int i=0,j=len(s)-1;
    while(i<j){ if(s[i]!=s[j]) return 0; i++; j--; }
    return 1;
}

void replace_char(char*s, char oldc, char newc){
    for(int i=0;s[i]!='\0';i++) if(s[i]==oldc) s[i]=newc;
}

void freq(const char*s, int f[256]){
    for(int i=0;i<256;i++) f[i]=0;
    for(int i=0;s[i]!='\0';i++) f[(unsigned char)s[i]]++;
}

int main(){
    char s1[200], s2[200];
    int ch;
    printf("Enter first string: ");
}

```

```

fgets(s1,200,stdin); // keep newline possibly
printf("Enter second string: ");
fgets(s2,200,stdin);

do{
    printf("\n1.Count vowels\n2.Check palindromes\n3.Replace
char\n4.Frequency of each char\n5.Exit\nChoice: ");
    scanf("%d",&ch); getchar(); // consume newline
    if(ch==1){
        int c1=0,c2=0;
        for(int i=0;s1[i]!='\0';i++) if(is_vowel(s1[i])) c1++;
        for(int i=0;s2[i]!='\0';i++) if(is_vowel(s2[i])) c2++;
        printf("Vowels: s1=%d, s2=%d\n",c1,c2);
    }else if(ch==2){
        printf("s1 is %spalindrome\n", is_pal(s1)?"":"not ");
        printf("s2 is %spalindrome\n", is_pal(s2)?"":"not ");
    }else if(ch==3){
        char o,nw;
        printf("Replace which char? "); o=getchar(); getchar();
        printf("With which char? "); nw=getchar(); getchar();
        replace_char(s1,o,nw); replace_char(s2,o,nw);
        printf("s1: %s", s1); printf("s2: %s", s2);
    }else if(ch==4){
        int f1[256],f2[256]; freq(s1,f1); freq(s2,f2);
        printf("Freq in s1 (non-zero):\n");
        for(int i=0;i<256;i++) if(f1[i]) printf("%c:%d ",
(i>=32&&i<127)?i:'. ', f1[i]);
        printf("\nFreq in s2 (non-zero):\n");
        for(int i=0;i<256;i++) if(f2[i]) printf("%c:%d ",
(i>=32&&i<127)?i:'. ', f2[i]);
        printf("\n");
    }
}while(ch!=5);
return 0;
}

```

Q5. Dynamic Memory Allocation, Sum and Standard Deviation

- Concept:
 - malloc/calloc allocate memory at runtime on heap; free releases it.
 - Avoid memory leaks; check NULL returns; use correct size with sizeof.

```

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(){
    int n; printf("Enter count: "); scanf("%d",&n);
    if(n<=0){ printf("Invalid n"); return 0; }
    double *a = (double*)malloc(n*sizeof(double));
    if(!a){ printf("Allocation failed"); return 0; }
    for(int i=0;i<n;i++){ printf("a[%d]: ",i); scanf("%lf",&a[i]); }
    double sum=0; for(int i=0;i<n;i++) sum+=a[i];
    double mean=sum/n, var=0;
    for(int i=0;i<n;i++){ double d=a[i]-mean; var+=d*d; }
    var/=n; // population variance
}

```

```

    double sd=sqrt(var);
    printf("Sum=%.4f Mean=%.4f SD=%.4f\n", sum, mean, sd);
    free(a);
    return 0;
}

```

Q6. Structure Employee and Operations

```

#include <stdio.h>
#include <string.h>

#define MAX 100
typedef struct {
    int emp_id;
    char name[50];
    char dept[30];
    double salary;
} Employee;

void input(Employee e[], int *n){
    printf("Enter number of employees: "); scanf("%d", n); getchar();
    for(int i=0;i<*n;i++){
        printf("Emp %d ID: ",i+1); scanf("%d",&e[i].emp_id); getchar();
        printf("Name: "); fgets(e[i].name,50,stdin);
        e[i].name[strcspn(e[i].name,"\n")]=0;
        printf("Department: "); fgets(e[i].dept,30,stdin);
        e[i].dept[strcspn(e[i].dept,"\n")]=0;
        printf("Salary: "); scanf("%lf",&e[i].salary); getchar();
    }
}

void display(Employee e[], int n){
    for(int i=0;i<n;i++){
        printf("%d | %s | %s | %.2f\n", e[i].emp_id, e[i].name, e[i].dept,
e[i].salary);
    }
}

void highest(Employee e[], int n){
    double mx = (n>0)?e[0].salary:0;
    for(int i=1;i<n;i++) if(e[i].salary>mx) mx=e[i].salary;
    printf("Employees with highest salary (%.2f):\n", mx);
    for(int i=0;i<n;i++) if(e[i].salary==mx)
        printf("%d %s %s %.2f\n", e[i].emp_id, e[i].name, e[i].dept,
e[i].salary);
}

void avg_dept(Employee e[], int n){
    for(int i=0;i<n;i++){
        int counted=0;
        for(int k=0;k<i;k++) if(strcmp(e[k].dept,e[i].dept)==0){ counted=1;
break; }
        if(counted) continue;
        double sum=0; int cnt=0;
        for(int j=0;j<n;j++) if(strcmp(e[j].dept,e[i].dept)==0){
sum+=e[j].salary; cnt++; }
        printf("Dept %s Avg Salary = %.2f\n", e[i].dept, sum/cnt);
    }
}

int main(){
    Employee e[MAX]; int n=0;
    input(e,&n);
    display(e,n);
}

```

```

    highest(e,n);
    avg_dept(e,n);
    return 0;
}

```

Q7. File I/O: Read, Compute Pass/Fail, Write to Another File

- Operations used:
 - fopen, fscanf/fgets, fprintf, fclose; check NULL; text mode.
- Policy: pass if marks ≥ 40 (example).

```

#include <stdio.h>
#include <stdlib.h>

typedef struct { int roll; char name[50]; float marks; } Stu;

int main(){
    FILE *in=fopen("students.txt","r");
    if(!in){ printf("Cannot open input\n"); return 0; }
    FILE *out=fopen("results.txt","w");
    if(!out){ printf("Cannot open output\n"); fclose(in); return 0; }
    Stu s;
    // Expected line format: roll name marks (name without spaces or use
    fgets+parse)
    while(fscanf(in, "%d %49s %f", &s.roll, s.name, &s.marks)==3){
        const char* res = (s.marks>=40.0)?"PASS":"FAIL";
        fprintf(out, "Roll:%d Name:%s Marks:%.2f Result:%s\n", s.roll,
s.name, s.marks, res);
    }
    fclose(in); fclose(out);
    printf("Results written to results.txt\n");
    return 0;
}

```

Q8. Header Files, Macros, Conditional Compilation

- Header files: collections of declarations/definitions (functions, macros, types) included using #include to share interfaces and enable modularity.
- Program:

```

#include <stdio.h>

// Macros
#define C2F(c) ((c)*9.0/5.0 + 32.0)
#define F2C(f) (((f)-32.0)*5.0/9.0)

// Enable one of them
// #define ENABLE_C2F
#define ENABLE_F2C

int main(){
#ifdef ENABLE_C2F

```

```

    double c; printf("Celsius: "); scanf("%lf",&c);
    printf("Fahrenheit=%.2f\n", C2F(c));
#endif
#ifdef ENABLE_F2C
    double f; printf("Fahrenheit: "); scanf("%lf",&f);
    printf("Celsius=%.2f\n", F2C(f));
#endif
    return 0;
}

```

- Significance of preprocessor directives:
 - Handle inclusion (#include), macros (#define), conditional compilation (#if/#ifdef), improving portability, configuration, and performance before compilation.